Final Project: Paradise Nursery Shopping Application



Estimated time needed: 1 hour

Introduction

In this final project, you will create a shopping cart application for an online plant shop which offers a variety of house plants.

The *Paradise Nursery* shopping cart features will include:

- A Landing page with a button linking to the product listing page
- A navigation bar with links to the landing, product listing, and shopping cart pages
- A card for each plant that showcases the different plants along with their images, name, description, cost and an **Add to cart** button.
- A minimum of two sections describing the plants in that section. For example, "Aromatic Plants" and "Medicinal Plants".
- A cart page which displays the products in the cart.
- The cart should have a card for each type of plant in the cart. Each card should have the thumbnail, the unit cost, the cost for all of the plants of that type and buttons to increase and decrease the quantity along with **Delete** button.
- A **Continue Shopping** and **Checkout** buttons

You will apply the knowledge and skills gained from the practice project to implement dynamic functionalities in the final project. Hence it is recommended to complete the practice project before attempting the final project. For example, features like

The content of this lab is licensed under Apache 2.0

displaying the cart quantity in the navbar icon and updating the total cost when the user changes the item quantity will be part of the final project.

Note:

1. Please ensure that all you files are updated and pushed to the public github repository and your deployed URL is live and publically accessible.

2. As mentioned in the *"Final Project Overview"*, you can submit your project deliverables through either Option 1: AI-Graded Submission and Evaluation or Option 2: Peer-Graded Submission and Evaluation.

3. **Option 1: AI-Graded Submission and Evaluation**
   Submit the **public GitHub repository URLs** for the following project files: README.md, AboutUs.jsx, App.css, App.jsx, CartSlice.jsx, ProductList.jsx, and CartItem.jsx. Your project code must be stored in a **public GitHub repository named** `e-plantShopping`.

4. **Option 2: Peer-Graded Submission and Evaluation**
   Submit the **deployed application URL** and the **public GitHub repository URL** for peer evaluation.When you deploy, make a note of the **live application URL** Please ensure-

- All project files must be **committed and pushed** before submission.
- Your application must be **deployed** and the **live URL** should be available.
- You may host your app using **GitHub Pages** or any other hosting platform of your choice.

Learning objectives

After completing this lab, you will be able to:

- React Components: Create functional React components using component composition and nesting.
- State Management with Hooks: Implement React Hooks, specifically the useState

The content of this lab is licensed under Apache 2.0

control the visibility of elements.

- Redux Integration: Integrate Redux within an application using Redux concepts like actions, reducers, and the store.
- Rendering Dynamic Data: Dynamically render data fetched from an array of objects into the UI. You will map over arrays to generate lists of components.
- Handling Events and Conditional Rendering: Handle user events such as button selection and trigger corresponding actions.

Prerequisites

- Basic knowledge GitHub and your own GitHub account
- Understanding of React function components, props, hooks and React Redux Toolkit
- Web browser with a console (such as Chrome DevTools or Firefox Console)
- It is mandatory to complete the Practice Project

The content of this lab is licensed under Apache 2.0

Important notice about this lab environment

Skills Network Cloud IDE (based on Theia and Docker) is an open-source IDE (Integrated Development Environment) that provides an environment for hands-on labs in course and project-related labs.

Please be aware that sessions for this lab environment are not persistent. Every time you connect to this lab, a new environment is created for you. You will lose data if you exit the environment without saving to GitHub or another external source. Plan to complete these labs in a single session to avoid losing your data.

The content of this lab is licensed under Apache 2.0

Setting up your environment in GitHub

1. Fork the repository

You need to fork the GitHub repository for your React application. The GitHub repository with the skeleton code for this project is here.Please use the same repository name when creating your fork:

- https://github.com/ibm-developer-skills-network/e-plantShopping.git
- After following the link above, click on the fork button.



2. This repository contains the basic layout of the React application for this project.
3. You will use this forked repository to push your latest code to keep a record of the work you do. Periodically save all your files. Your files must be saved to perform git commands.
4. Perform `git add`, `git commit`, and `git push` commands to update changes from your application folder to your GitHub repository for proper code management.
5. As part of the peer review project (if you choose this option) , you will need to deploy your app so your peers can review the UI. You can review these instructions for assistance with GitHub and for deployment instructions of react application.

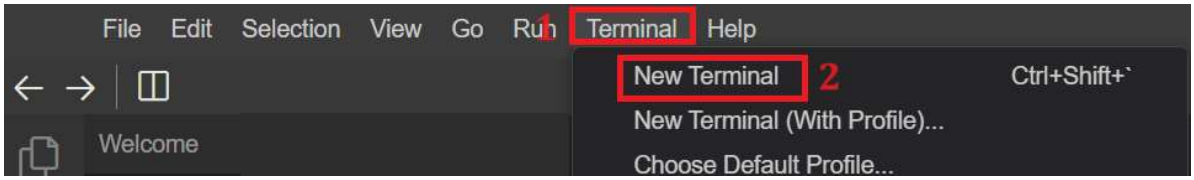
The content of this lab is licensed under Apache 2.0

If you've logged out and want to resume work on the project, clone the forked repository where you previously pushed the code. After cloning, you can use `git push origin` without the need to execute `git remote add...` to push your changes directly.

The content of this lab is licensed under Apache 2.0

Create your React project

1. If you do not have an open terminal, select the "Terminal" tab at the top-right of the window, and then select "New Terminal". The screenshot below shows you where to locate these options on your screen.



2. Now clone the forked repository using given command by replacing `<forked-repo-link>` with your own repository link.

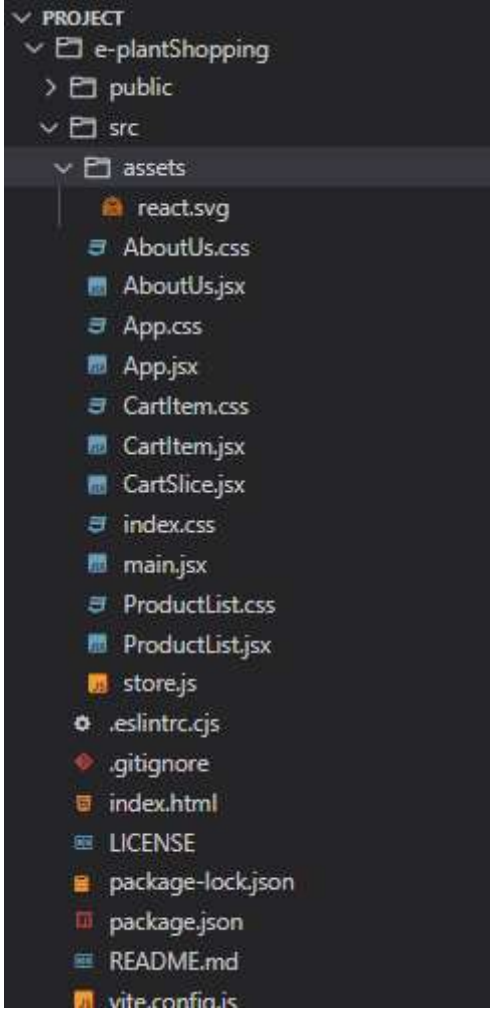
```
git clone <forked-repo-link>
```

Note: You should keep the original repository name e-plantShopping

3. Make sure your application name matches your project.

4. After cloning the repository you will see the folder structure like given screenshot.

The content of this lab is licensed under Apache 2.0



5. Write the command to enter the application folder in the terminal. The command will set your terminal path to run the React application in the `<forked-folder-name>` folder.

```
cd e-plantShopping
```

Run

6. To make sure that the code you have cloned is working correctly, you need to follow the given steps:

    - Write the following command in the terminal and select Enter to install all the necessary packages to execute the application.

```
npm install
```

Run

The content of this lab is licensed under Apache 2.0

- Then execute the following command to run the application, providing you with port number 4173.

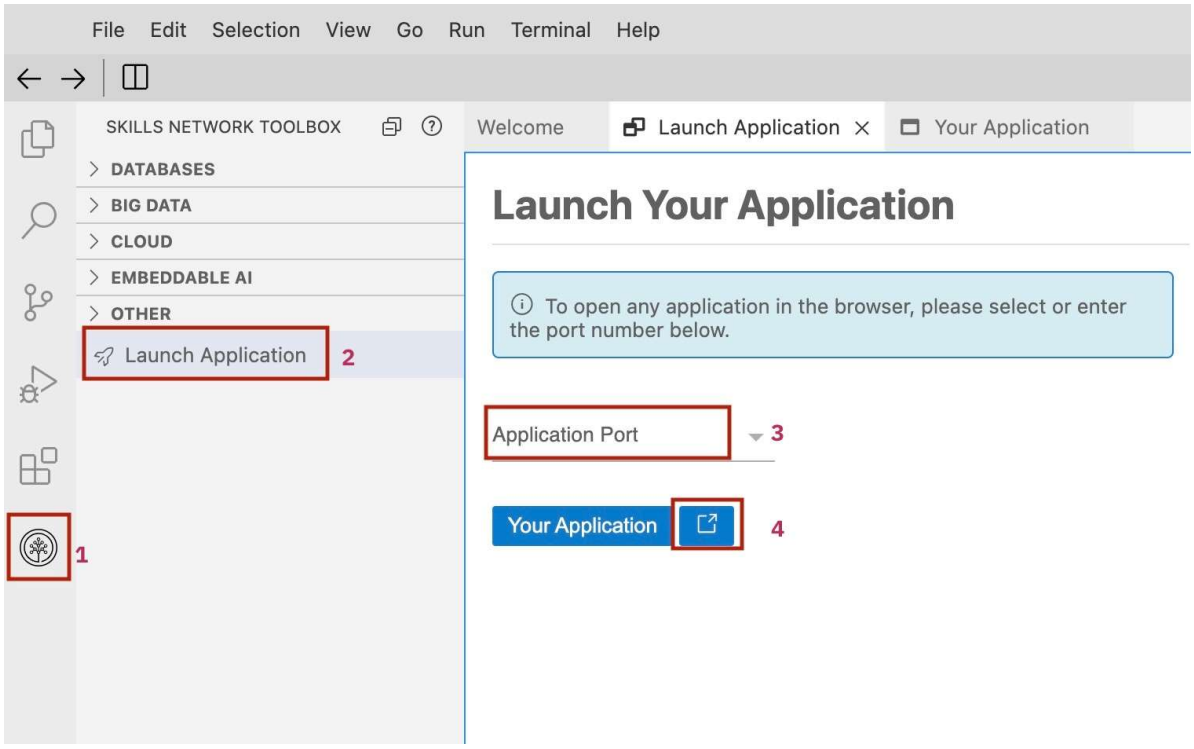
```
npm run preview
```

Run

7. To view your React application, click the Skills Network icon on the left panel (refer to number 1). This action will open the **Skills Network Toolbox**. Next, click **Launch Application** (refer to number 2). Enter port number **4173** in **Application Port** (refer to number 3) and click

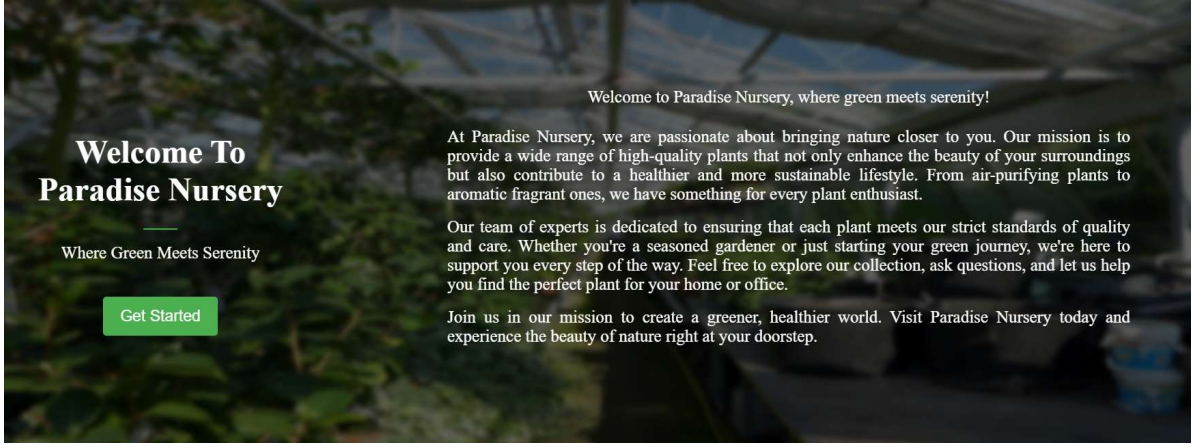


.

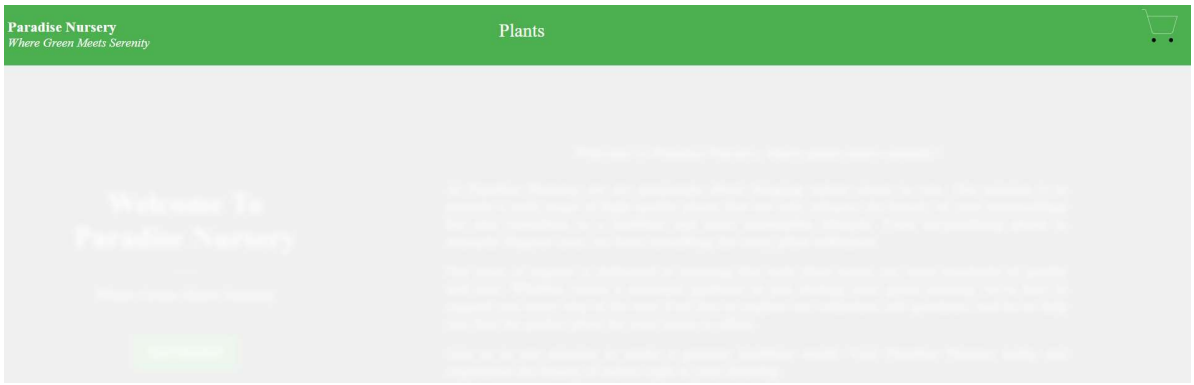


8. The output will be according to given screenshot with background picture.

The content of this lab is licensed under Apache 2.0



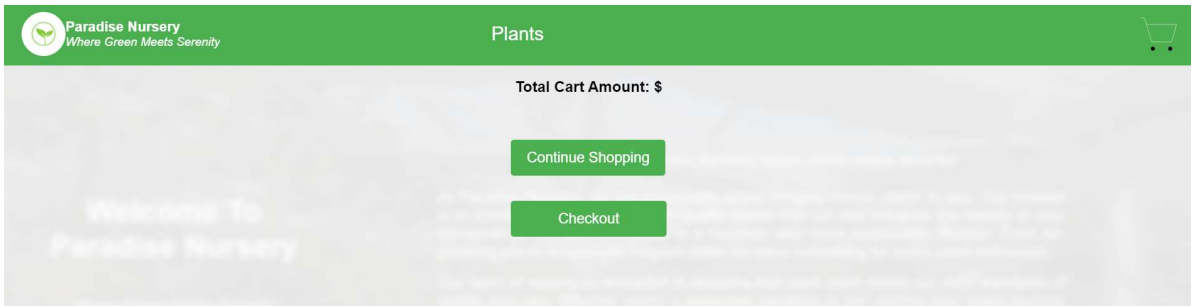
9. Now click on `Get Started` button and then you will see the given layout according to screenshot which includes navbar with green background color.



10. The navbar contains three links:

    - `Paradise Nursey` - This will take you back to the landing page of the application.
    - `Plants` - This will navigate you to the page where information related to page will be visible.
    - `Cart icon` - This will navigate you to the cart items section.

11. When you will click on `cart icon` output will be visible as per given screenshot.



The content of this lab is licensed under Apache 2.0

Complete the required tasks explained on the pages that follow. You will assessed based on these tasks.

The content of this lab is licensed under Apache 2.0

Updating the Readme.md file

Open your README.md file in your GitHub repository.

Includes the repository name e-plantShopping in the contents of the file.You can also mention the project name and provides a brief overview of your application

Note : Make sure the file includes the repository name e-plantShopping in the contents.

Make sure that you save these changes by pushing your code to your GitHub repository.

`Note : This step can be skipped if you choose to proceed with the Peer Graded Assignment.`

The content of this lab is licensed under Apache 2.0

Task 1: ProductList component Layout

The product page will allow your users to shop for the different plants you sell. Each plant will display on its own "card" with its related data stored in the plant object. You will store the plant objects in an array. Follow these steps for the array and plant objects.

1. Display the Plant Array

- Navigate to the `ProductList.jsx` component and you will see an array named `plantsArray` with the plants details.
- Each plant object contains the categories, properties name, image URL, description, and cost.

2. Display Plant Details within `div` tag with class name **product-grid**.

- Utilize array methods to map over the plant array.

> *Hint: use the `map()` method to iterate array.*

- Render each plant's details on the page, including name, image, description, and cost.

3. Display an **Add to Cart** button for each plant.

▶ **Display plants and add to cart button solution**

- Include above code within class name **product-grid**.

4. Create one variable named `addedToCart` for state management using the **useState** hook to track which products are added to the cart.

The content of this lab is licensed under Apache 2.0

5. Add to Cart Functionality

- Create the `handleAddToCart` function to implement the functionality for adding a plant to the cart when the user selects the **Add to Cart** button. This function should take one parameter that contains the information of the selected plant. This information should then be dispatched to the addItem inside the function component `CartSlice`.

- Additionally, reflect the product has been added to the cart. Update the `setAddedToCart` state to by setting the product name as a key and its value to true.

▶ Sample solution for add to cart

> *Note: Make sure that you `import` the `addItem` reducer from `CartSlice.jsx`*

6. The `handleAddToCart()` function will carry the details of that plant which user want to add in the cart. And the plant details to the cart at a global level using `CartSlice.jsx`.

7. Make sure that you save these changes by pushing your code to your GitHub repository.

The content of this lab is licensed under Apache 2.0

Task 2: State management using Redux

1. You have the basic layout in the `CartSlice.jsx` file.

2. Define Reducer Functions

- Now implement the reducer property of the slice for adding, removing, and updating the number of items in the cart.

- These reducer functions will be called when user wants to add or remove the quantity of plants within the `cartItems` component.

- The `addItem()` reducer adds a new plant item to the `items` array which you initialized in the previous step.

- The `addItem()` function should get called when the user selects an **Add to cart** on the plant listing page. Subsequently, the `handleAddToCart()` gets called which has the plant type as a parameter.

- The `handleAddToCart()` function will then dispatch the plant details to the `addItem()` reducer function in `CartSlice.jsx`.

▶ addItem() reducer sample solution

- Now you need to complete code for the `removeItem()` and `updateQuantity()` reducers.

- `removeItem()`: This reducer removes an item from the cart based on its name and gets called when the user wants to remove products from the cart.

▶ removeItem() reducer sample solution

The content of this lab is licensed under Apache 2.0

- `updateQuantity()`: To create this function, start by extracting the item's name and amount from the `action.payload`. Then, look for the item in the `state.items` array that matches the extracted name. If the item is found, update its quantity to the new amount provided in the payload. This ensures the item's quantity is correctly updated based on the action.

▶ updateQuantity() reducer sample solution

3. Handle Actions

- Export the action creators, `addItem()` to use in `ProductList.jsx`, `removeItem()`, and `updateQuantity()`, to use in the `CartItem.jsx`.
- Also export the reducer as the default to use in `store.js`.

4. Make sure you save these changes by pushing your code to your GitHub repository.

The content of this lab is licensed under Apache 2.0

Task 3: CartItems component

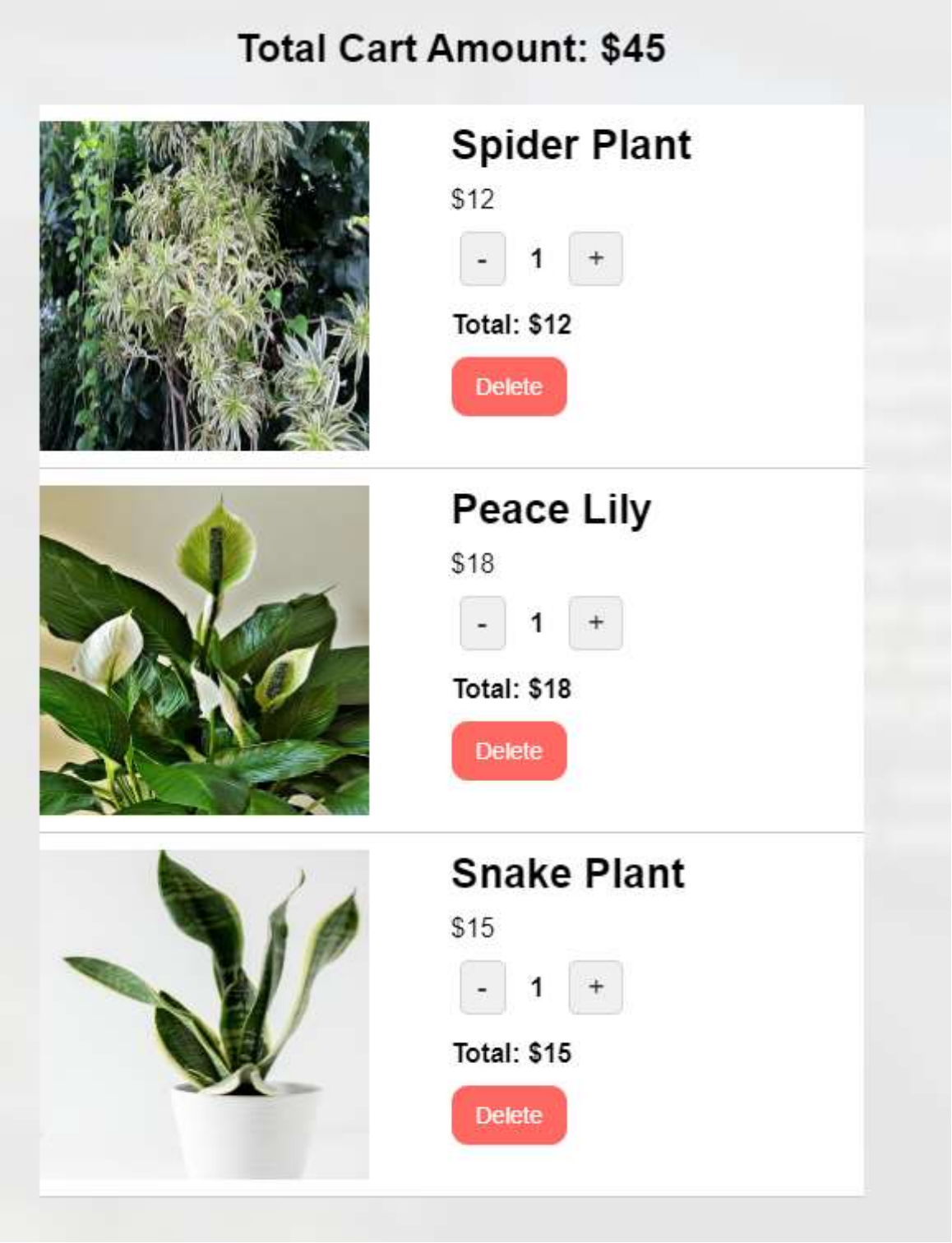
Next, you will complete the development of the `CartItem.jsx` component which displays the items held in the shopping cart. This component has a number of functionalities that you find in a typical shopping cart:

- Calculate the total for all items in the cart.
- Calculate the subtotal for each plant type in the cart.
- Continue shopping
- Increment and decrement the number of each plant type in the cart
- Remove (delete) a plant type from the cart altogether.

You will dispatch the increment, decrement, and update quantity details from a Redux file.

When you are done, the cart page should look similar to the following:

The content of this lab is licensed under Apache 2.0



1. Cost of all items in cart

- In the `calculateTotalAmount()` you need a function to calculate the cost of all of the items in the cart. There are a number of ways you can calculate this.

▶ Sample algorithm description for calculating the total cost

2. Continue shopping

The content of this lab is licensed under Apache 2.0

- Users should be able to return to the plant listing page to continue shopping while on the shopping cart page. So, in the `handleContinueShopping()` function, call the `onContinueShopping(e)` function passed from the parent component.

3. Checkout

- In this project, you are not required to provide the `handleCheckoutShopping()` function, but you may wish to for further exploration and practice. For now, just add in the following code to alert the user this function will get added at a later date.

```
const handleCheckoutShopping = (e) => {
  alert('Functionality to be added for future reference');
};
```

4. Increment and decrement

- For the `handleIncrement()` and `handleDecrement()` functions, you need to dispatch the `updateQuantity()` reducer in the `CartSlice.jsx` file. In the function argument, either add one to the `item.quantity` value or subtract one, respectively.

- Also, for the `handleDecrement()` you will need an if-else to handle the case. -- If the item's quantity is greater than 1, dispatch `updateQuantity` to decrease the quantity by 1. -- Else if the quantity would drop to 0, dispatch the `removeItem` action to remove the plant type from the cart.

▶ Click here to see hints

5. Remove plant from the cart

- For the `handleRemove()` function you need to dispatch the `removeItem` action to delete the item from the cart.

6. Item subtotal

- Calculate the total cost for an item by multiplying its quantity with its unit price in

The content of this lab is licensed under Apache 2.0

- Extract the numeric value from the item's cost string using `parseFloat(item.cost.substring(1))` before performing the multiplication.

> *Note: Ensure that these event handlers update the UI in real time. When the user changes the number of a plant type, the following should update accordingly: • The individual plant quantity. • The item's subtotal. • The overall total cost. • The the total number of items in the cart icon. • After a user clicks the "Add to Cart" button for a plant item, the button should become disabled and grayed out, and its label should update to "Added to Cart" to indicate that the item has already been added.*



7. Make sure that you save these changes by pushing your code to your GitHub repository.

The content of this lab is licensed under Apache 2.0

Task 4: Integrate Redux functionality in your components

1. ProductList Component

- Initialize the cart state within the Redux store to keep track of cart items.

▶ Click here to see hints

- Use the `addItem` action to add selected products to the cart.
- Access the Redux store to retrieve and display the total quantity of items currently in the cart.

▶ Click here to see hints

2. CartItem Component

- Use the `updateQuantity` action to change how many items are in the cart.
- Use the `addItem` action to add a new product to the cart.
- Use the `removeItem` action to delete an item completely from the cart.

▶ Click here to see hints

Note: Make sure that you save these changes by pushing your code to your GitHub repository

The content of this lab is licensed under Apache 2.0

Task 5: Import details to store.js

1. Importing Necessary Functions and Files:

    - The `configureStore()` function from the `@reduxjs/toolkit` package is imported to set up the Redux store.
    - The `cartReducer` from the `CartSlice.jsx` file which is imported, manages the state slice related to the shopping cart.

```
import { configureStore } from '@reduxjs/toolkit';
import cartReducer from './CartSlice';
```

2. Configuring the Store:

    - The `configureStore()` function is used to setup the Redux store.
    - Inside the configuration object passed to `configureStore()`, the `reducer` key specifies the reducer functions. In this case, the `cartReducer` is assigned to manage the `cart` slice of the state.

```
// Create a Redux store using configureStore from Redux Toolkit
const store = configureStore({
    // Define the root reducer object
    reducer: {
        // 'cart' is the name of the slice in the store, and it's managed by
        cart: cartReducer,
    },
});

export default store; // Export the store for use in the app (e.g., in <Provi
```

3. Exporting the Store:

    - The configured Redux store is exported using `export default store;`, so it can be used throughout the application to manage state.

The content of this lab is licensed under Apache 2.0

```
export default store;
```

Note: - This store.js code is pre-configured in the repository and ready for use. - Make sure that you save these changes by pushing your code to your GitHub repository

The content of this lab is licensed under Apache 2.0

Task 6: Set up the global store

1. Navigate to the `main.jsx` file in the `src` folder.
2. The `Provider` component from the `react-redux` library is already imported. This component enables all components in the application to access the Redux store.

```
import { Provider } from 'react-redux';
```

3. The Redux store is imported from the `store.js` file. This store holds the application's state, using the reducer defined in the `CartSlice.jsx` file.

```
import store from './store.js';
```

4. The `App` component is wrapped with the `Provider` component, with the Redux store passed as a prop. This allows all components in the app to access and interact with the global state managed by Redux.

```
<Provider store={store}>
  <App />
</Provider>
```

5. Deploying Your Application with GitHub Pages

▶ Deploy using GitHub Pages

Note:

- This code is pre-configured in the repository and ready for use.
- Make sure that you save these changes by pushing your code to your GitHub repository

The content of this lab is licensed under Apache 2.0

- **After deploying on GitHub Pages, it may take some time for all contents and images to appear properly. Please wait a few extra minutes for the application to load completely.**

The content of this lab is licensed under Apache 2.0

Project Completion

Congratulations!

You completed this project! Great work.

- If you have not already done so, deploy your application. You can deploy it using GitHub Pages or your own hosting site.
- Ensure you have pushed all the updated files to your github repository

Summary

- You created function React components, composed, and nested them.
- You implemented React Hooks, specifically the `useState` and `useEffect` hooks to manage component-level states.
- You integrated Redux within your React application and applied Redux concepts like actions, reducers, and the store.
- You dynamically rendered data fetched from an array of objects onto the UI. You mapped over arrays to generate lists of components.
- You handled user events such as button selection and triggered corresponding actions to add and remove items in the cart.

The content of this lab is licensed under Apache 2.0

Checklist for Submission

Follow the checklist below to verify that your project meets all requirements before submission.

Submit your work through either **Option 1: AI-Graded Submission and Evaluation** or **Option 2: Peer-Graded Submission and Evaluation**, depending on the submission path you choose for project evaluation.

Checklist for Option 1: AI-Graded Submission and Evaluation

After completing the final project, you should:

1. **Submit the GitHub URL of the `README.md` file** containing the project name details.
    - Ensure the content includes the repository name `e-plantShopping`.
2. **Submit the GitHub URL of the `AboutUs.jsx` file**, which includes details about the company.
3. **Submit the GitHub URL of the `App.css` file**, which includes the background image for the **Paradise Nursery** landing page.
4. **Submit the GitHub URL of the `App.jsx` file**, which includes the **Paradise Nursery** landing page with the company's welcome message and a **"Get Started"** button.
5. **Submit the GitHub URL of the `CartSlice.jsx` file**, which includes the Redux slice for the shopping cart.
    - Ensure it defines Redux functionality to **add, remove, and update** cart items.
6. **Submit the GitHub URL of the `ProductList.jsx` file**, which implements the product listing page with the following functionalities:

The content of this lab is licensed under Apache 2.0

- Display **at least six unique houseplants per category**, each with a thumbnail, name, and price.
- Group the plants into **at least three categories**.
- Include **"Add to Cart"** buttons that:
  - Add the product to the cart.
  - Disable the button after the item is added.
  - Update the cart.
- Include a **navbar** visible on both Product Listing and Cart pages, with links to **Home**, **Plants**, and **Cart**.
- Display a **cart icon** showing the **total number of items dynamically**.

7. **Submit the GitHub URL of the `CartItem.jsx` file**, which displays the **Shopping Cart page** with the following functionalities:

   - Show the **total cart amount** for all plants.
   - Display the **total cost per plant** in the cart.
   - Each cart item should display the **thumbnail, plant name, and unit price**.
   - Include buttons to **increase or decrease quantity**, updating totals correctly.
   - Include a **delete button** to remove items from the cart.
   - Include a **checkout button** that displays **"Coming Soon"** or a similar message.
   - Include a **continue shopping button** that links back to the **Product Listing** page.

Before You Submit

- Ensure all files are **updated, committed, and pushed** to the `e-plantShopping` repository.
- Each submitted link must be a **public GitHub URL** pointing **directly to the specific file**.
- Verify that all functionalities work correctly before final submission.

The content of this lab is licensed under Apache 2.0

Checklist for Option 2: Peer Review Evaluation

After completing the final project, you should:

1. **Submit the URL of your deployed application**, hosted on **GitHub Pages** or another platform.
    - Ensure your app is fully functional and **publicly accessible**.
2. **Submit the GitHub URL of your public repository**, named `e-plantShopping`.
    - Ensure all files are committed and pushed before submission.

Project Evaluation Rubric for peer review

GitHub Link:

- Public GitHub repository URL
- Redux-related files and code implemented correctly

By URL of your deployed application: Landing Page

- Background image displayed properly
- Includes a paragraph about the company
- Displays the company name
- Includes a "Get Started" button that links to the product listing page

Product Listing Page

- Displays at least six unique houseplants for sale, each showing a thumbnail, name, and price
- Groups plants into at least three categories
- Includes an "Add to Cart" button for each plant that:
    - Adds the selected plant to the shopping cart
    - Increments the shopping cart icon count

The content of this lab is licensed under Apache 2.0

Header

- Displays on both the product listing and shopping cart pages
- Includes a shopping cart icon that dynamically displays the total number of items
- Allows navigation between the product listing and shopping cart pages

Shopping Cart Page

- Displays the total number of plants in the cart
- Displays the total cost of all items
- Each plant type displays a thumbnail, name, and unit price
- Includes buttons to increase or decrease the quantity of each plant and update totals accordingly - Includes a delete button to remove items from the cart
- Includes a checkout button displaying "Coming Soon" or a similar message
- Includes a "Continue Shopping" button linking back to the product listing page

Before You Submit

- Confirm that your deployed application link opens correctly.
- Ensure your GitHub repository is public and includes all the latest updates.

Author

Richa Arora

© IBM Corporation. All rights reserved.

Checklist for Submission

The content of this lab is licensed under Apache 2.0

Follow the checklist below to verify that your project meets all requirements before submission.

Submit your work through either **Option 1: AI-Graded Submission and Evaluation** or **Option 2: Peer-Graded Submission and Evaluation**, depending on the submission path you choose for project evaluation.

Checklist for Option 1: AI-Graded Submission and Evaluation

After completing the final project, you should:

1. **Submit the GitHub URL of the `README.md` file** containing the project name details.
    - Ensure the content includes the repository name `e-plantShopping`.
2. **Submit the GitHub URL of the `AboutUs.jsx` file**, which includes details about the company.
3. **Submit the GitHub URL of the `App.css` file**, which includes the background image for the **Paradise Nursery** landing page.
4. **Submit the GitHub URL of the `App.jsx` file**, which includes the **Paradise Nursery** landing page with the company's welcome message and a **"Get Started"** button.
5. **Submit the GitHub URL of the `CartSlice.jsx` file**, which includes the Redux slice for the shopping cart.
    - Ensure it defines Redux functionality to **add, remove, and update** cart items.
6. **Submit the GitHub URL of the `ProductList.jsx` file**, which implements the product listing page with the following functionalities:
    - Display **at least six unique houseplants per category**, each with a thumbnail, name, and price.
    - Group the plants into **at least three categories**.
    - Include **"Add to Cart"** buttons that:

The content of this lab is licensed under Apache 2.0

    - Add the product to the cart.
    - Disable the button after the item is added.
    - Update the cart.
  - Include a **navbar** visible on both Product Listing and Cart pages, with links to **Home**, **Plants**, and **Cart**.
  - Display a **cart icon** showing the **total number of items dynamically**.

7. **Submit the GitHub URL of the `CartItem.jsx` file**, which displays the **Shopping Cart page** with the following functionalities:

  - Show the **total cart amount** for all plants.
  - Display the **total cost per plant** in the cart.
  - Each cart item should display the **thumbnail, plant name, and unit price**.
  - Include buttons to **increase or decrease quantity**, updating totals correctly.
  - Include a **delete button** to remove items from the cart.
  - Include a **checkout button** that displays **"Coming Soon"** or a similar message.
  - Include a **continue shopping button** that links back to the **Product Listing** page.

Before You Submit

- Ensure all files are **updated, committed, and pushed** to the `e-plantShopping` repository.
- Each submitted link must be a **public GitHub URL** pointing **directly to the specific file**.
- Verify that all functionalities work correctly before final submission.

Checklist for Option 2: Peer Review Evaluation

After completing the final project, you should:

1. **Submit the URL of your deployed application**, hosted on **GitHub Pages** or another

The content of this lab is licensed under Apache 2.0

- Ensure your app is fully functional and **publicly accessible**.

2. **Submit the GitHub URL of your public repository**, named `e-plantShopping`.

    - Ensure all files are committed and pushed before submission.

Project Evaluation Rubric for peer review

GitHub Link:

- Public GitHub repository URL
- Redux-related files and code implemented correctly

By URL of your deployed application: Landing Page

- Background image displayed properly
- Includes a paragraph about the company
- Displays the company name
- Includes a "Get Started" button that links to the product listing page

Product Listing Page

- Displays at least six unique houseplants for sale, each showing a thumbnail, name, and price
- Groups plants into at least three categories
- Includes an "Add to Cart" button for each plant that:
    - Adds the selected plant to the shopping cart
    - Increments the shopping cart icon count
    - Disables the button after the plant is added

Header

- Displays on both the product listing and shopping cart pages
- Includes a shopping cart icon that dynamically displays the total number of items
- Allows navigation between the product listing and shopping cart pages

The content of this lab is licensed under Apache 2.0

- Displays the total number of plants in the cart
- Displays the total cost of all items
- Each plant type displays a thumbnail, name, and unit price
- Includes buttons to increase or decrease the quantity of each plant and update totals accordingly - Includes a delete button to remove items from the cart
- Includes a checkout button displaying "Coming Soon" or a similar message
- Includes a "Continue Shopping" button linking back to the product listing page

Before You Submit

- Confirm that your deployed application link opens correctly.
- Ensure your GitHub repository is public and includes all the latest updates.

Author

Richa Arora

© IBM Corporation. All rights reserved.

← Project Completion

The content of this lab is licensed under Apache 2.0